

MAEG4060 Virtual Reality Systems and Applications

Course Project – Final Report

Project title: Interactive Physical Simulation of Off-Road Vehicle Dynamics in a Mountainous Terrain

Group members:

Guo Tsun Hei (1155186272): 3D Modeling, Texturing, Rigging, and Animation

Yim Fu Chong (1155176974): Programming, Physics Engine, and System Integration

Submission date: 11 April 2026

Submission deadline: 17 April 2026

Abstract

Brief summary of the interactive 3D environment (not a game: no missions or scoring), main interaction model, and how the submission satisfies the course requirements (DirectX 9.0c, Visual C++, keyboard/gamepad, physics). *[Write 150–250 words when drafting.]*

Contents

1	Introduction	3
1.1	Project scope	3
1.2	Alignment with course requirements	3
1.3	Mapping to marking criteria (group project)	3
2	Completed application	3
2.1	Feature overview	3
2.2	Executable and source layout	3
2.3	Screenshots	3
3	User guide	4
3.1	System requirements	4
3.2	Installation and running	4
3.3	Controls	4
3.3.1	Keyboard	4
3.3.2	Gamepad	4
3.4	Troubleshooting	4
4	Technical description	4
4.1	Architecture overview	4
4.2	DirectX 9 rendering	5
4.3	Terrain and model navigation	5
4.3.1	Bilinear interpolation for terrain height	5
4.3.2	Surface normal alignment	5
4.4	Animation	5
4.5	Collision detection and response	5

4.5.1	Axis-aligned bounding boxes (AABB)	5
4.5.2	Bounding spheres	5
4.5.3	Optional dynamic interaction	5
4.6	Physics and simulation	5
4.7	Environmental effects	6
4.8	Bonus work	6
5	3D models: sources and modifications	6
6	Conclusion	7
7	References	7
A	Appendix A: Build configuration	7
B	Appendix B: Third-party libraries	7
C	Appendix C: User Guide (standalone PDF)	8

1 Introduction

1.1 Project scope

Describe the virtual environment theme and what the user can do. Relate this to the Stage 1 proposal and note any intentional changes from the original plan.

1.2 Alignment with course requirements

The application is implemented in **Visual C++** with **DirectX 9.0c**. It is a technical demonstration / sandbox (**not** a video game: no missions, no score). Input: **keyboard** and **gamepad**. The scene includes **physical simulation** (e.g. gravity, vehicle dynamics, terrain interaction, collisions).

1.3 Mapping to marking criteria (group project)

Table 1 lists Stage 3 items from the course project description; use the technical sections below to make each item easy for markers to find.

Table 1: Where to address each marked component (typical section pointers — adjust after writing).

Component (weight)	See section	Notes
Gamepad / joystick (5%)	§3.3	XInput / analog mapping
Model navigation (5%)	§4.3	Terrain following, camera or vehicle
Collision, static objects (10%)	§4.5	e.g. AABB / spheres
Physics-based motion (10%)	§4.6	Forces, integration, friction
Maya animation in VR (20%, group)	§4.4	.X / ID3DXAnimationController etc.
User guide (5%)	§3	Controls, requirements, run instructions
Report (15%)	Whole document	Clarity, completeness
Bonus (10%)	§4.8	Moving-object collision, extra effects, ...

2 Completed application

2.1 Feature overview

Enumerate implemented features (terrain, vehicle, props, day/night, HUD, audio if any, ...).

2.2 Executable and source layout

State what is submitted (e.g. Release executable, solution path, asset folders). Do not embed large binaries in the PDF; refer to the submission package.

- Main entry point / project file: {e.g. main.cpp, .sln}
- Key modules: *{list}*
- Asset pipeline: *{heightmap, textures, .x models}*

2.3 Screenshots

Include a few figures of the running application. Below: placeholders; replace with `\includegraphics` when ready.

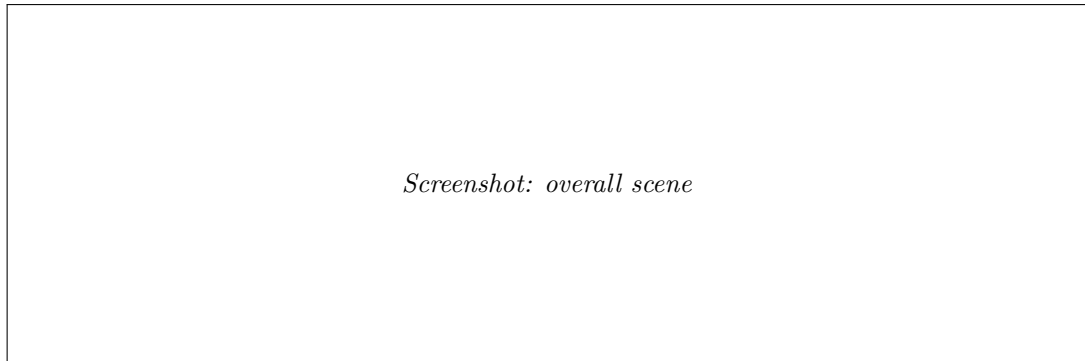


Figure 1: Caption the view (e.g. vehicle on terrain, time of day).

3 User guide

3.1 System requirements

OS version, DirectX / runtime expectations, display resolution, optional gamepad.

3.2 Installation and running

Steps to build (if source) or run the executable; working directory if assets must be beside the .exe.

3.3 Controls

3.3.1 Keyboard

Input	Action
W / A / S / D	<i>{describe}</i>
...	...

3.3.2 Gamepad

Describe XInput mapping: sticks, triggers, face buttons. Explain throttle / steer if applicable.

3.4 Troubleshooting

Common issues (missing DLL, assets not found, fullscreen, ...).

4 Technical description

This section summarizes the main techniques used in the DirectX 9.0c (fixed-function pipeline) implementation: terrain sampling, vehicle–terrain coupling, collision choices, animation, lighting, and environmental effects. The application is a physics sandbox (not a scored game).

4.1 Architecture overview

The real-time loop follows input → simulation → rendering: user input (keyboard via DirectInput, gamepad via XInput) drives vehicle and camera logic; the simulation updates positions, collisions, and animation time; Direct3D 9 draws the scene with depth buffering and texturing.

4.2 DirectX 9 rendering

The renderer uses a Direct3D 9 device with a depth-stencil buffer, textured meshes, and directional lighting. Meshes and animations are loaded from the `.X` format; the coordinate system is left-handed, consistent with `D3DXMatrixLookAtLH` and standard DirectX samples.

4.3 Terrain and model navigation

4.3.1 Bilinear interpolation for terrain height

The terrain height at an arbitrary world position (x, z) is obtained from a heightmap grid. To avoid discrete “stair-stepping” when the vehicle moves, the engine samples the four neighbouring heightmap texels and applies **bilinear interpolation**:

$$h(x, z) = (1 - t_x)(1 - t_z) h_{00} + t_x(1 - t_z) h_{10} + (1 - t_x)t_z h_{01} + t_x t_z h_{11},$$

where $t_x, t_z \in [0, 1]$ are fractional coordinates within the cell. This yields a continuous height field suitable for stable contact and visual alignment.

4.3.2 Surface normal alignment

The vehicle’s up vector is aligned with the **terrain surface normal**, computed from the local geometry (e.g. cross product of edges on the heightfield triangle under the vehicle). Pitch and roll follow the normal while yaw is driven by steering input, giving kinematic terrain-following without penetrating the mesh.

4.4 Animation

Maya keyframed animation (e.g. windmill sails) is exported in the `.X` format and played with `ID3DXAnimationController`. Vehicle wheels use **procedural** rotation tied to simulated linear speed ($v \approx \omega r$) in addition to any exported hierarchy.

4.5 Collision detection and response

4.5.1 Axis-aligned bounding boxes (AABB)

Rectangular props (e.g. wooden crates) use **AABB** tests: fast intersection in world space along x , y , and z extents. Response can slide or push the vehicle along the minimum penetration axis.

4.5.2 Bounding spheres

Irregular objects (e.g. boulders) use **bounding sphere** collision: distance between sphere centres compared to the sum of radii. Spheres are cheap to update when objects rotate or roll and pair well with impulse-based response for loose rocks.

4.5.3 Optional dynamic interaction

Where implemented, **dynamic-to-dynamic** contact allows the vehicle to impart momentum to loose boulders so they roll on slopes and interact with other objects (see §4.8).

4.6 Physics and simulation

Gravity is applied as a constant downward acceleration ($g \approx 9.81$ in engine units per second squared). The integrator updates linear velocity and position each frame; **rolling friction** and braking reduce speed when throttle is released.

4.7 Environmental effects

A **day/night cycle** can orbit the sun light using trigonometric functions and adjust clear colour and light colours for time of day. **Texture splatting** can blend grass and rock materials based on terrain slope (e.g. rock above a threshold angle). An on-screen HUD (`ID3DXFont`) may show speed, position, and cycle state.

4.8 Bonus work

Dynamic-to-dynamic collision and momentum transfer for loose boulders (and any other bonus items) are summarized here if implemented; if none, state “No bonus features claimed.”

5 3D models: sources and modifications

Per course requirements, external assets are listed with Sketchfab links and licenses (aligned with the Stage 1 proposal). **Modifications** should be updated to match what was actually done in Maya and the engine (retopo, materials, wheel hierarchy, scale, UVs, animation export, ...).

1. Half Dome Terrain Landscape

- Author: Studio Lab
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: <https://sketchfab.com/3d-models/half-dome-terrain-landscape-86ef075dc7d9414583ac2180e1e651a3>
- Usage: Reference for high-resolution terrain geometry and heightmap generation.
- Modifications: *{describe: e.g. heightmap resolution, splatting masks, scale in world units}*

2. Controllable Vehicle (ATV)

- Title: ATV from game “PURE”
- Author: barking_dogo
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: <https://sketchfab.com/3d-models/atv-from-game-pure-f2e454611ebd4151b9d7aa31815c35e4>
- Usage: Main controllable vehicle model (wheel nodes separated for procedural animation).
- Modifications: *{describe: e.g. hierarchy, textures, export to .X}*

3. Rock Boulder (Game Ready Asset)

- Author: Aparicio Silva 3D
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: <https://sketchfab.com/3d-models/rock-boulder-game-ready-asset-0079cb0e717c4aeebe6bca21ec21b0d9>
- Usage: Dynamic physics / bounding-sphere collision.
- Modifications: *{describe: e.g. scale, collision radius tuning}*

4. Windmill

- Author: KIFIR
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: <https://sketchfab.com/3d-models/windmill-fe13862b9e3c4dc887d10e297d86f17f>
- Usage: Keyframed animation and environmental aesthetics.
- Modifications: *{describe: e.g. Maya keyframes, AnimationSet in .X}*

5. Wooden Crate

- Author: Domingos Studios
- License: Creative Commons Attribution (CC BY 4.0)
- Source: Sketchfab
- URL: <https://sketchfab.com/3d-models/wooden-crate-3c0259d17f3b4306890cdc809b545670>
- Usage: Static collision (AABB).
- Modifications: *{describe: e.g. placement, AABB extents}*

6 Conclusion

Lessons learned, known limitations, possible future work.

7 References

- Course project description (MAEG4060, 2025–26 Semester 2).
- DirectX SDK / MSDN documentation.
- External assets: cite consistently with §5.

A Appendix A: Build configuration

Compiler version, Windows SDK, preprocessor defines, Debug vs. Release notes.

B Appendix B: Third-party libraries

If any (e.g. math helpers), list versions and licenses.

C Appendix C: User Guide (standalone PDF)

The following pages reproduce the submitted user guide (`User_Guide.pdf`): controls (keyboard and gamepad), hardware and software requirements, launch instructions, and troubleshooting.

MAEG4060 Course Project User Guide

Interactive Physical Simulation of Off-Road Vehicle Dynamics

Guo Tsun Hei; Yim Fu Chong

April 2026

1 How to start the application

- Open the `Game_Build` folder in your submission package.
- **Double-click** the main application executable (`.exe`) in that folder to launch the program.
- Keep the executable in the same folder structure as submitted so relative paths to assets (models, textures, heightmaps) resolve correctly. If the program reports missing files, run it from `Game_Build` without moving only the `.exe` away from its data folders.

2 Hardware and software requirements

- **Platform:** Windows PC (64-bit Windows recommended).
- **Graphics API:** The application uses **DirectX 9.0c**. You need a DirectX 9-compatible GPU and the **DirectX 9 End-User Runtime** (or equivalent system components) installed as required by your Windows version.
- **Display:** A desktop resolution of at least **1280×720** is recommended; other resolutions are supported when the window is resized.
- **Optional input:** An **XInput-compatible gamepad** (e.g. Xbox controller) for analog control. Keyboard control works without a gamepad.

3 Controls

3.1 Keyboard (WASD and related keys)

Input is handled via the keyboard using a typical first-person / vehicle-style layout (DirectInput), for example:

Input	Action
W / S	Forward / reverse (throttle)
A / D	Steer left / right
Space	Brake (or hand brake, depending on build)
Esc	Quit or return (if implemented)

Exact key bindings may match the on-screen HUD or readme in the binary package; the layout above reflects the project design (WASD movement with separate brake).

3.2 Gamepad (XInput)

With an XInput controller connected, the application maps analog axes and triggers for smoother control than the keyboard:

Control	Action
Left stick	Steering
Right trigger	Throttle (accelerate)
Left trigger	Brake / reverse (as implemented)
Start / Back	Pause or UI (if implemented)

Triggers provide analog throttle similar to a real vehicle; keyboard W/S act as digital throttle inputs.

4 Troubleshooting

- **Missing DLLs or DirectX errors:** Install or repair the DirectX 9 end-user runtime and ensure GPU drivers are up to date.
- **Black screen or failed device creation:** Try windowed mode, update graphics drivers, and confirm no other application has exclusive full-screen access to the GPU.
- **Assets not found:** Run from `Game_Build` with the supplied `Assets` (or equivalent) folder beside the executable.
- **Gamepad not detected:** Use a wired connection or official wireless adapter; confirm the controller appears in Windows as an XInput device.